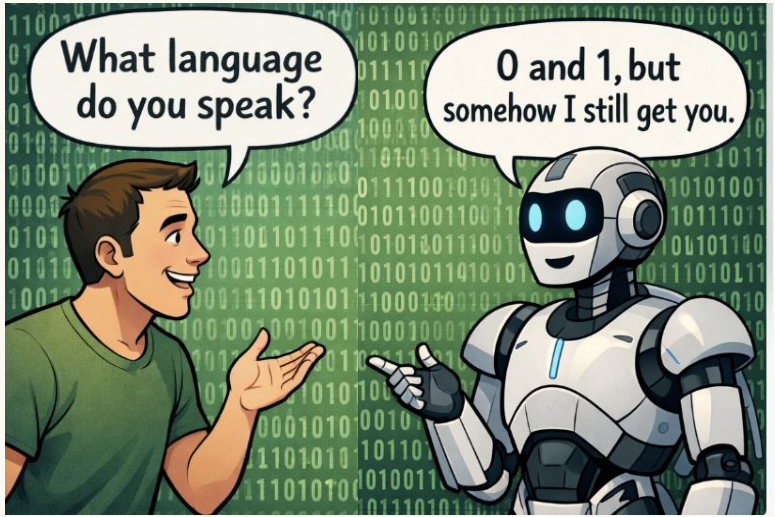
Bag of Words and Vector Representations

Table of contents

- Why Do We Need Vector Representations?
- How to go from text to vectors?
- One-hot Encoding
- Bag of Words (BoW)
- BoW Limitations
- Vector Similarity with Pytorch
- Dense Representations: Word Embeddings
- Word Embeddings with Pytorch
- Sparsity in PyTorch
- Word2Vec Training Objectives
- Contextual Embeddings (Transformers)
- Embeddings Usage in Real World

Why Do We Need Vector Representations?

- Machines cannot process the data the way we do.
- They transform the data into numbers.




```
Ramadan Kareem to everyone  ->   01010100001111010101101010 1…
```

How to go from text to vectors?

- Tokenization (27th March session):

  `Ramadan Kareem to everyone  -> ["Ramadan", "Kareem", "to", "everyone"]`

- Map the tokens into numbers:

  `["Ramadan", "Kareem", "to", "everyone" ]  -> [... , ... , ... , ... ]`

  - How to choose the numbers?

One-hot Encoding

- **Single-word** Representation
- Let Vocabulary be $V = \{w_1, \ldots, w_{|V|}\}$
- One-hot Encoding for a token (word) w_j is $\mathbf{e}_j \in \mathbb{R}^{|V|}$ with $(\mathbf{e}_j)_i = 1[i = j]$.

- Example
  - Vocabulary: ["Ramadan", "Kareem", "to", "everyone"]
  - "Ramadan" -> [1 , 0 , 0 , 0]
  - "Kareem" -> [0 , 1 , 0 , 0]
  - "to" -> [0 , 0 , 1 , 0]
  - "everyone" -> [0 , 0 , 0 , 1]

Bag of Words (BoW)

- Represent a **document** by the count of each word appearing in it.
- For a Document D and a Vocabulary V: $V = \{w_1, \ldots, w_{|V|}\}$
- A sentence x is $\mathbf{x} \in \mathbb{R}^{|V|}$ with $x_i = \#\{w \in D : w = w_i\}$.

Sentence 01: "`The month of Ramadan is here! Ramadan is a blessed month`"

Sentence 02: "`Ramadan Kareem`"

Vocab: `["The", "month", "of", "Ramadan", "is", “here”, “a”, “blessed”, “Kareem”, “!”]`

Vector 01: `[ 1 , 2 , 1 , 2 , 2 , 1 , 1 , 1 , 0 , 1 ]`

Vector 02: `[ 0 , 0 , 0 , 1 , 0 , 0 , 0 , 0 , 1 , 0 ]`

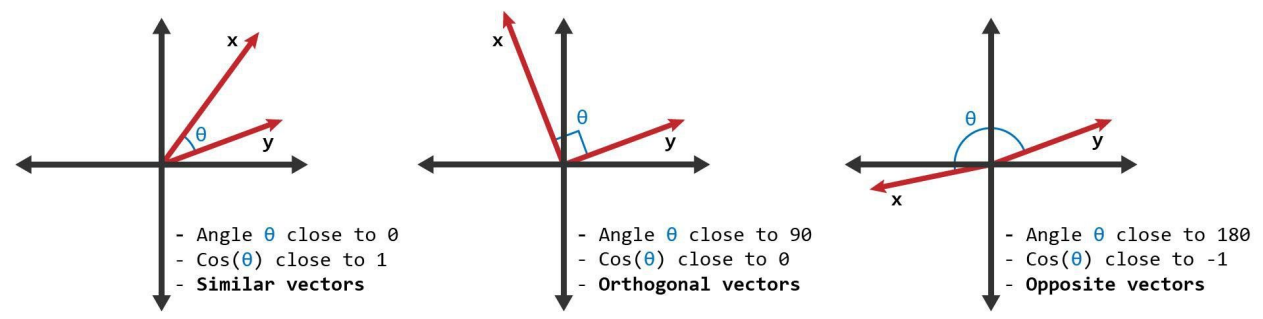
BoW Limitations

- High dimensional vectors:
  - Each word = dimension
  - Sentence = point in high-D space
  - With 50k vocab → 50k-dimensional vectors!
- Sparse Representations
  - Most entries are zero
  - Sentence of 10 words in 50,000-dim vector → 49,990 zeros
- Lack of meaningful representations
  - In the space, two completely different sentences -> close vectors.
    - Vocab: `["The", "month", "I", "Ramadan", "love", “hate”, “a”, “blessed”, “cats”]`
    - I love cats -> `[ 0 , 0 , 1 , 0 , 1 , 0 , 0 , 0 , 1 ]`
    - I hate cats -> `[ 0 , 0 , 1 , 0 , 0 , 1 , 0 , 0 , 1 ]`

Vector Similarity with Pytorch

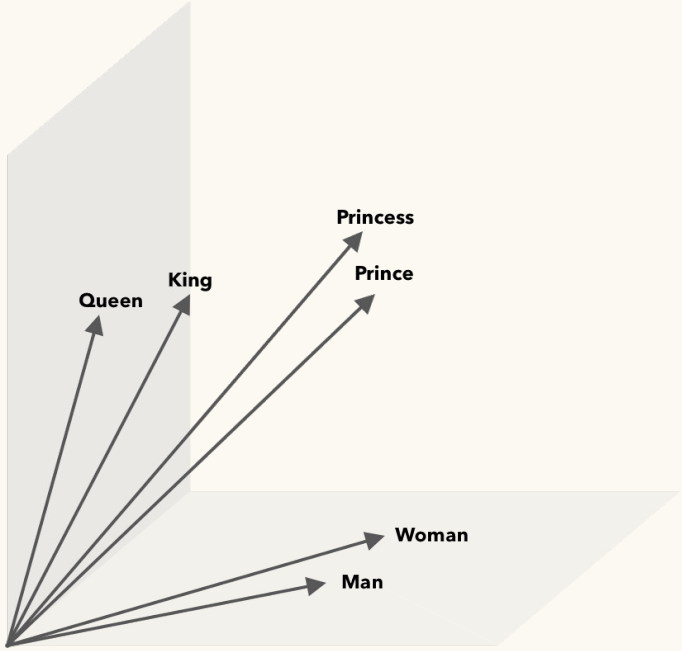
```
import torch

x = torch.tensor([0., 0., 1., 0., 1., 0., 0., 0., 1.])

y = torch.tensor([0., 0., 1., 0., 0., 1., 0., 0., 1.])

cos = torch.nn.functional.cosine_similarity(x, y, dim=0)

euclid = torch.dist(x, y, p=2)

print(f"cosine={cos:.3f}, euclidean={euclid:.3f}")
```

```
cosine=0.667,
euclidean=1.414
```



Dense Representations: Word Embeddings

- Word2Vec, GloVe changed NLP (around 2013-2014)
- Each word → small dense vector (e.g. 100D)
- Learned, not handcrafted
- Similar words → Close vectors
- e.g.: Women-Man ≈ Queen - King



Dense Representations: Word Embeddings Example

Embedding Dimension = Number of features
d

Vocab Size
|V|

	Gender	Wealth	Royalty	Weight	Speak
King	1	1	1	0.8	1
Queen	0	1	0.8	0.4	1
Man	1	0.3	0.2	0.6	1
Woman	0	0.3	0.2	0.5	1
Cat	1	0	0	0.1	0

Embeddings Weight Matrix -> |V| * d

Word Embeddings with Pytorch

```
import torch, torch.nn as nn


emb = nn.Embedding(num_embeddings=50_000, embedding_dim=128)

sentence_ids = torch.tensor([10, 500, 7777, 500])   # toy token ids

word_vectors = emb(sentence_ids) # [number_of_tokens × embedding_dim] -> [4 × 128] for example

sentence_vector = word_vectors.mean(dim=0)       # simple sentence embedding with mean pooling
```

"Ramadan" → [0.8, 0.1, 0.4]
"Kareem" → [0.7, 0.2, 0.5]
"to" → [0.1, 0.0, 0.0]
"everyone" → [0.6, 0.1, 0.3]

([0.8+0.7+0.1+0.6] / 4, [0.1+0.2+0.0+0.1] / 4, [0.4+0.5+0.0+0.3] / 4)
= [0.55, 0.10, 0.30]

Sparsity in PyTorch

- A dense tensor -> All values are stored, even if most of them are zeros
- A sparse tensor -> only non-zero elements and their indices are stored
  - `[0, 0, 5, 0, 0]` stored as `indices = [2], values = [5]`
- We can convert a dense vector to a sparse one by `dense.to_sparse()`

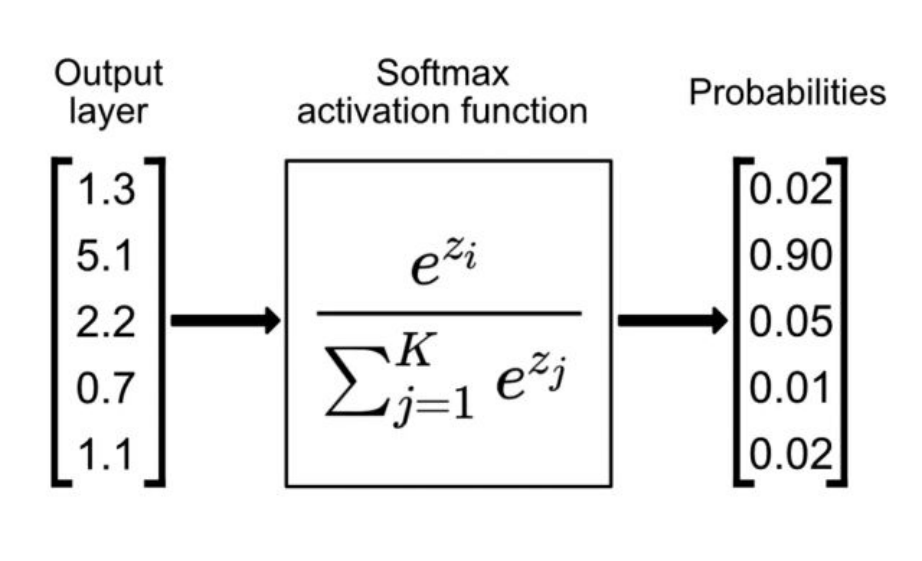
```
dense = torch.tensor([0, 0, 5, 0, 0])
sparse = dense.to_sparse()
print("Sparse indices:", sparse._indices())  # -> tensor([[2]])
print("Sparse values:", sparse._values())    # -> tensor([5])
print("Sparse size:",   sparse.size())       # -> torch.Size([5])
# Convert back to dense
back_to_dense = sparse.to_dense()
print("Back to dense:", back_to_dense)       # -> tensor([0, 0, 5, 0, 0])
```

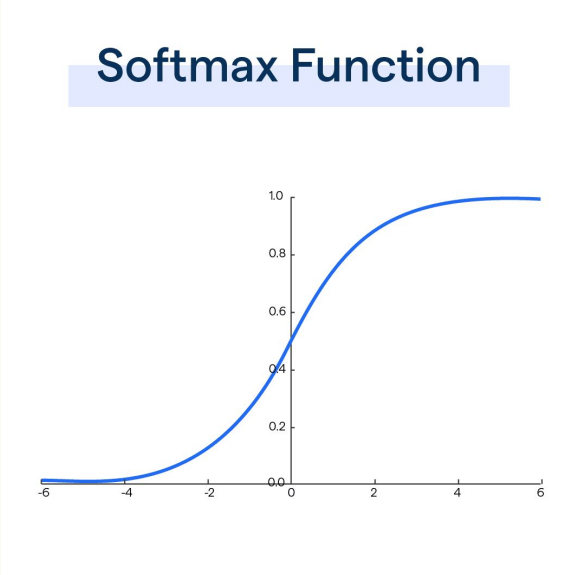
Embedding Sparsity in PyTorch

- Vocabulary size = **1 million words,** Embedding dim = **128,** training batch contains **300 token IDs**
- **Weight Matrix =** `[vocab_size × embedding_dim] = [1,000,000 × 128]`
- `emb = nn.Embedding(100000, 128, sparse=True)`
- With `sparse = True`, the training stores and updates the gradients only for the **300** rows.
- Without `sparse = True`, the backpropagation computes a full dense gradient matrix (mostly zero) -> `[1_000_000 × 128]`
- Only the **gradients** during backprop become sparse.
- Not all optimizers work with sparse tensors
  - Adam Optimizer uses dense gradients
- You can use `torch.optim.SparseAdam`or `torch.optim.Adagrad`

Softmax Function - Reminder-

Converts a list of raw scores (logits) into probabilities.





Word2Vec Training Objectives

1. **Continuous Bag of Words (CBOW)** Given the words around a missing word, predict the missing word.

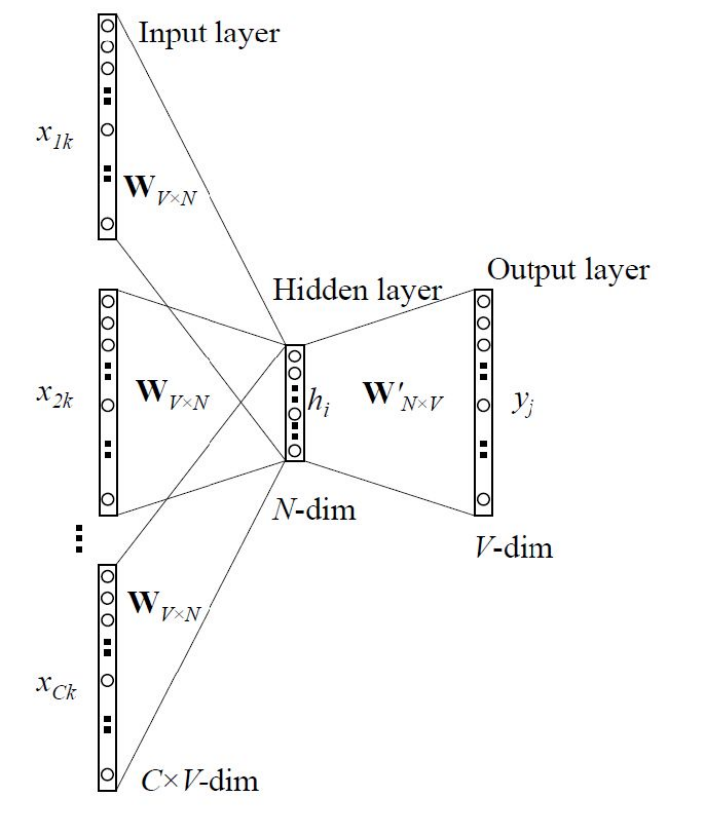
$$\prod_{t=1}^{T} P(w_t \mid \text{context}(w_t))$$

Example

- Context words: `"I ____ to school"`
- Input: `[I, to]`
- Output: `go`

⇩

1. Compute Embeddings of context words
2. Average embeddings
3. Feed into a classifier with Softmax
4. Output is the token with the highest probability



https://commons.wikimedia.org/wiki/File:Cbow.png

Word2Vec Training Objectives

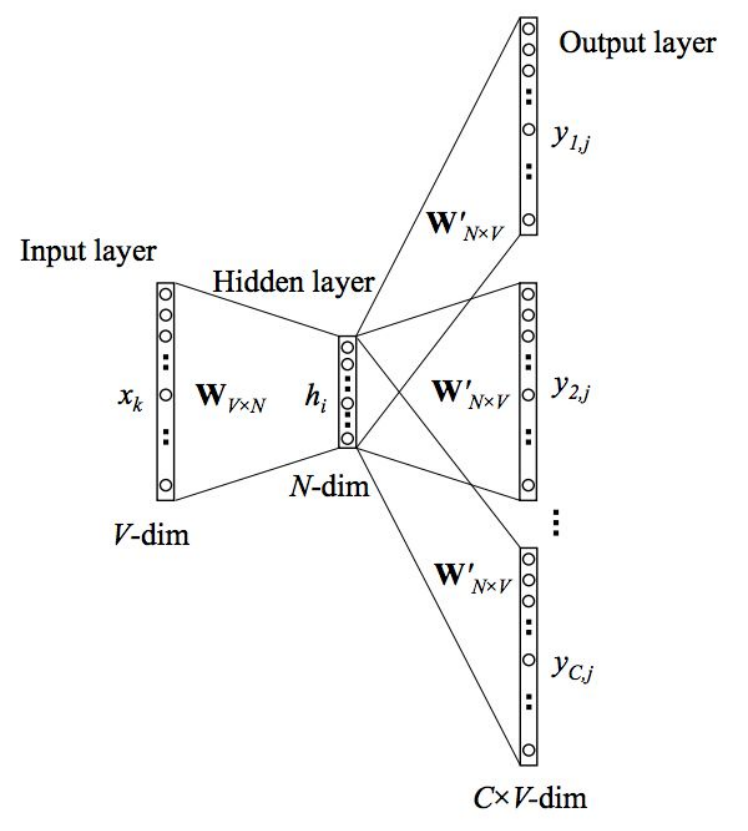
2. Skip-Gram: Given a single word, predict the words around it.

$$\prod_{t=1}^{T} \prod_{-m \le j \le m, j \ne 0} P(w_{t+j} \mid w_t)$$

Example

- Input: `go`
- Output: `[I, to, school]`

1. Compute Embeddings of the center word
2. Predict each context word with Softmax



https://commons.wikimedia.org/wiki/File:Skip-gram.png

Contextual Embeddings (Transformers)

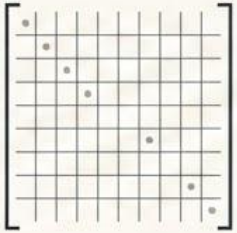
- With the appearance of Transformers models (2016-2018)
- The word gets different embedding based on the context.
- The word “can” has different vector representations
  - I can train an NLP model
  - I bought a can of tuna

How This Is Used in Real Tasks

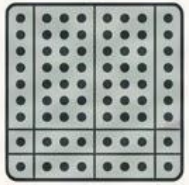
- Spam detection
- Sentiment analysis
- Search
- Recommendation
- RAG systems
- Document clustering

Summary

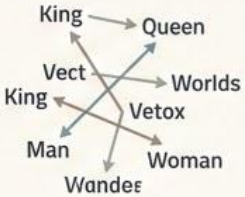
BoW
(Bag of Words)



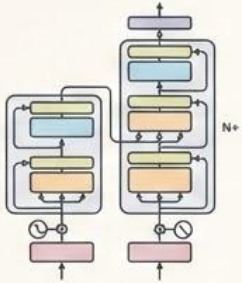
Sparse
Representations



Dense
Representations



Word2Vec/GloVe
(Word Embeddings)



Transformers
(Contextual
Embeddings)